41. Design Distributed Locking Service

Key Terms:

Term	Full Name	What it does
Mutex	Mutual Exclusion	Only one holder at a time
Lease	Time-limited lock	Auto-expires if holder crashes
Fencing Token	Monotonic counter	Prevents stale client from corrupting data
Redlock	Redis distributed lock	Majority quorum across Redis instances
TTL	Time To Live	Auto-expiration for leases
Heartbeat	Keep-alive signal	Extends lease while client is active
Raft/Paxos	Consensus algorithms	Replicated state machine for fault tolerance

Problem Statement

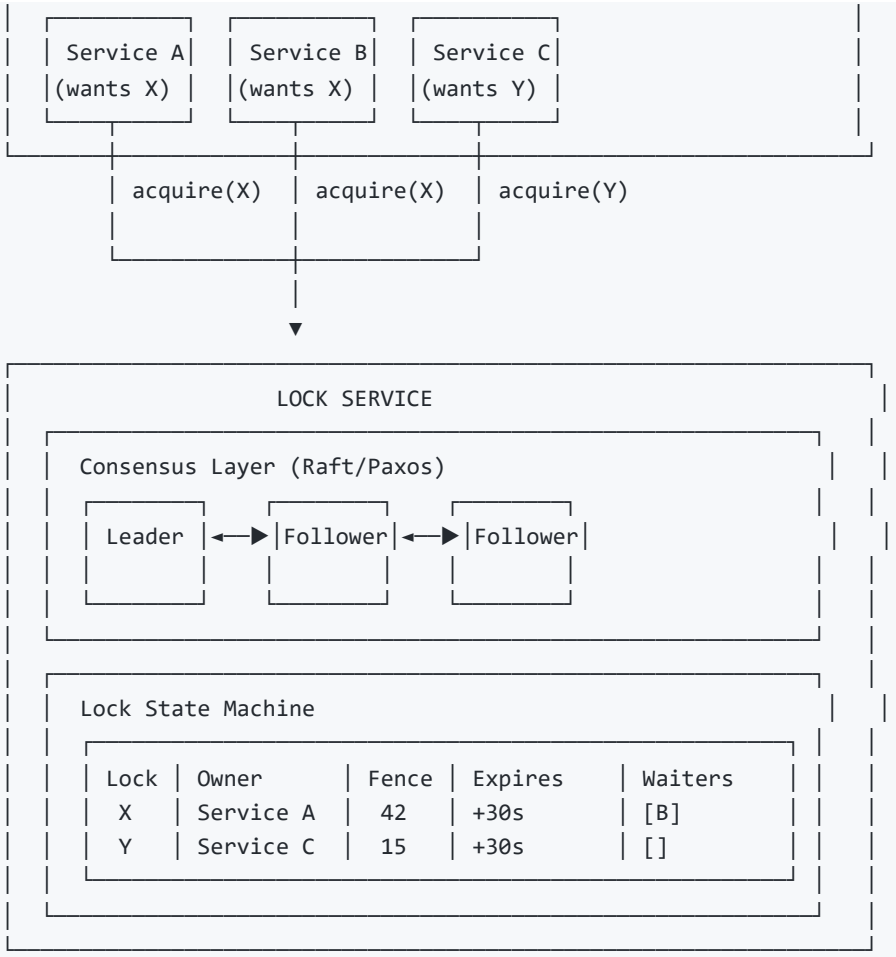
Design a distributed locking service that provides mutual exclusion across a cluster of servers.

Clarifying Questions to Ask

1. **Lock types**: Mutex, read-write, semaphore?
2. **Duration**: Short-lived or long-held locks?
3. **Fairness**: FIFO ordering or best-effort?
4. **Failure handling**: What if lock holder crashes?
5. **Scale**: How many locks? Lock operations per second?
6. **Clients**: In-datacenter or cross-region?
7. **Semantics**: Fencing tokens needed?

High-Level Design





API Design

Lock APIs

Method	Endpoint	Description
`POST`	`/api/v1/locks/acquire`	Acquire a lock
`POST`	`/api/v1/locks/release`	Release a lock
`POST`	`/api/v1/locks/renew`	Renew lock TTL (heartbeat)
`GET`	`/api/v1/locks/{lockKey}`	Get lock status

Acquire Lock Request

```
POST /api/v1/locks/acquire
{
  "lock_key": "order:123:processing",
  "owner_id": "worker-456",
  "ttl_seconds": 30,
```

```
  "wait_timeout_ms": 5000   // 0 = no wait (try-lock)
}

Response: 200 OK
{
  "acquired": true,
  "lock_key": "order:123:processing",
  "fence_token": 42,        // monotonically increasing
  "expires_at": "2025-03-15T10:00:30Z"
}

Response: 409 Conflict
{
  "acquired": false,
  "current_owner": "worker-789",
  "retry_after_ms": 500
}
```

Release Lock Request

```
POST /api/v1/locks/release
{
  "lock_key": "order:123:processing",
  "owner_id": "worker-456",
  "fence_token": 42          // must match
}
```

Admin APIs

Method	Endpoint	Description
`GET`	`/api/v1/locks/active`	List all active locks
`DELETE`	`/api/v1/locks/{lockKey}/force`	Force-release a lock (admin)

Key Components

Component	Purpose	Technology
Consensus Layer	Replicated state machine	Raft (etcd) / Paxos (Chubby)
Lock Manager	Track lock ownership	State machine
Lease Manager	Auto-expire stale locks	TTL with heartbeat
Watch Service	Notify waiters	Event subscription

Fencing Token	Prevent stale operations	Monotonic counter

Lock Operations

```
ACQUIRE LOCK:
┌─────────────────────────────────────────────────────────────────┐
│  1. Client sends: ACQUIRE(lock_id, client_id, ttl)              │
│  2. Leader checks lock state:                                   │
│     - If free: Grant lock, return fence_token                   │
│     - If held by same client: Extend TTL (reentrant)            │
│     - If held by other: Add to wait queue or reject             │
│  3. Replicate decision via Raft                                 │
│  4. Return result to client                                     │
└─────────────────────────────────────────────────────────────────┘

RELEASE LOCK:
┌─────────────────────────────────────────────────────────────────┐
│  1. Client sends: RELEASE(lock_id, client_id, fence_token)      │
│  2. Leader verifies ownership and fence token                   │
│  3. If valid: Release lock, grant to next waiter                │
│  4. Replicate via Raft                                          │
└─────────────────────────────────────────────────────────────────┘

HEARTBEAT (Keep-alive):
┌─────────────────────────────────────────────────────────────────┐
│  Client periodically sends heartbeat while holding lock         │
│  Server extends TTL on heartbeat                                │
│  If no heartbeat within TTL: Auto-release lock                  │
└─────────────────────────────────────────────────────────────────┘
```

Fencing Tokens

```
PROBLEM: Client thinks it has lock, but lock expired
┌─────────────────────────────────────────────────────────────────┐
│  Time 0: Client A acquires lock (token=42)                      │
│  Time 1: Client A pauses (GC, network delay)                    │
│  Time 2: Lock expires, Client B acquires (token=43)             │
│  Time 3: Client A resumes, thinks it has lock!                  │
│  Time 4: Both A and B write to resource → CORRUPTION            │
└─────────────────────────────────────────────────────────────────┘

SOLUTION: Fencing tokens
┌─────────────────────────────────────────────────────────────────┐
│  - Lock service returns monotonically increasing token          │
│  - Client includes token in all operations                      │
│  - Resource rejects operations with old tokens                  │
│                                                                 │
│  Time 4: A sends write with token=42                            │
│          Resource sees token=43 was already used                │
```

```
│          Resource rejects A's write → SAFE                      │
└─────────────────────────────────────────────────────────────────┘
```

Redis-Based Lock (Redlock)

```
REDLOCK ALGORITHM (for Redis without consensus):
┌─────────────────────────────────────────────────────────────────┐
│  5 independent Redis instances                                  │
│                                                                 │
│  Acquire:                                                       │
│  1. Get current time T1                                         │
│  2. Try SET lock_key client_id NX PX ttl on ALL 5 instances     │
│  3. Get current time T2                                         │
│  4. Lock acquired if:                                           │
│     - Majority (3+) instances returned OK                       │
│     - Total time (T2-T1) < ttl                                  │
│  5. If failed: Release on all instances                         │
│                                                                 │
│  Validity time = ttl - (T2-T1) - clock_drift                    │
└─────────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Partitioning**: Shard locks by key hash
- **Read scaling**: Serve reads from followers (for watch)
- **TTL tuning**: Balance between quick recovery and lease overhead
- **Batching**: Batch heartbeats for many locks
- **Caching**: Cache lock state locally with invalidation

Trade-offs

Decision	Option A	Option B	Recommendation
Backend	Redis	etcd/ZooKeeper	**etcd** - consensus built-in
Waiting	Block	Poll	**Watch** - efficient
Fairness	FIFO	Best-effort	**FIFO** - predictable
Fencing	Required	Optional	**Required** - safety

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Lock storage with Redlock algorithm for distributed mutual exclusion	Tech Guide
Consistent Hashing	Partitioning locks across nodes for scalability	Tech Guide
Raft Consensus	Ensuring lock state survives failures with replicated state machine	Tech Guide

How to Explain It Well

> **Opening**: "Distributed locks need consensus for correctness. I'll use Raft-based storage (etcd) with fencing tokens for safety."

> **Key talking points**:
>
> - "Raft consensus ensures lock state survives failures"
> - "TTL with heartbeat auto-releases locks from crashed clients"
> - "Fencing tokens prevent stale clients from corrupting resources"
> - "Watch API notifies waiters without polling"
> - "Redlock for Redis: majority quorum across independent instances"
